

Arquitetura de Computadores

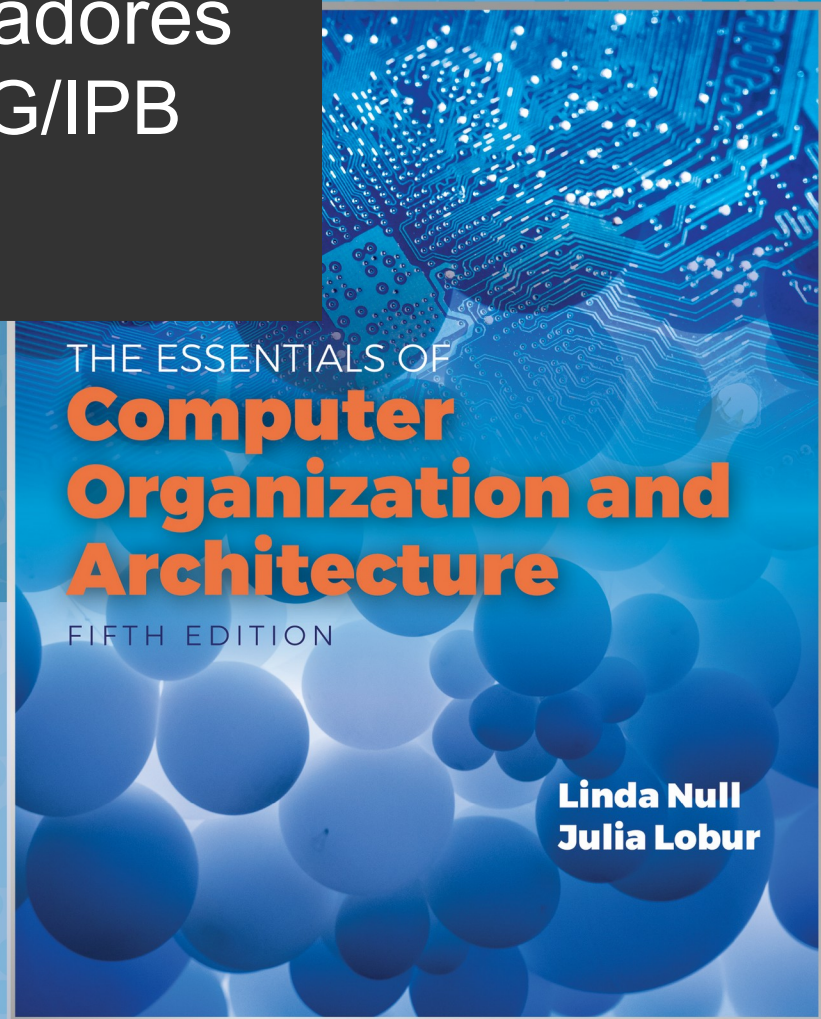
Eng. Informática, ESTiG/IPB

2025/2026

José Rufino

(baseado nos Capítulos 4 e 6 do livro “The Essentials of Computer Organization and Architecture” de Linda Null e Julia Lobur)

Unidade 5: Memória Principal e Cache da CPU



Objetivos

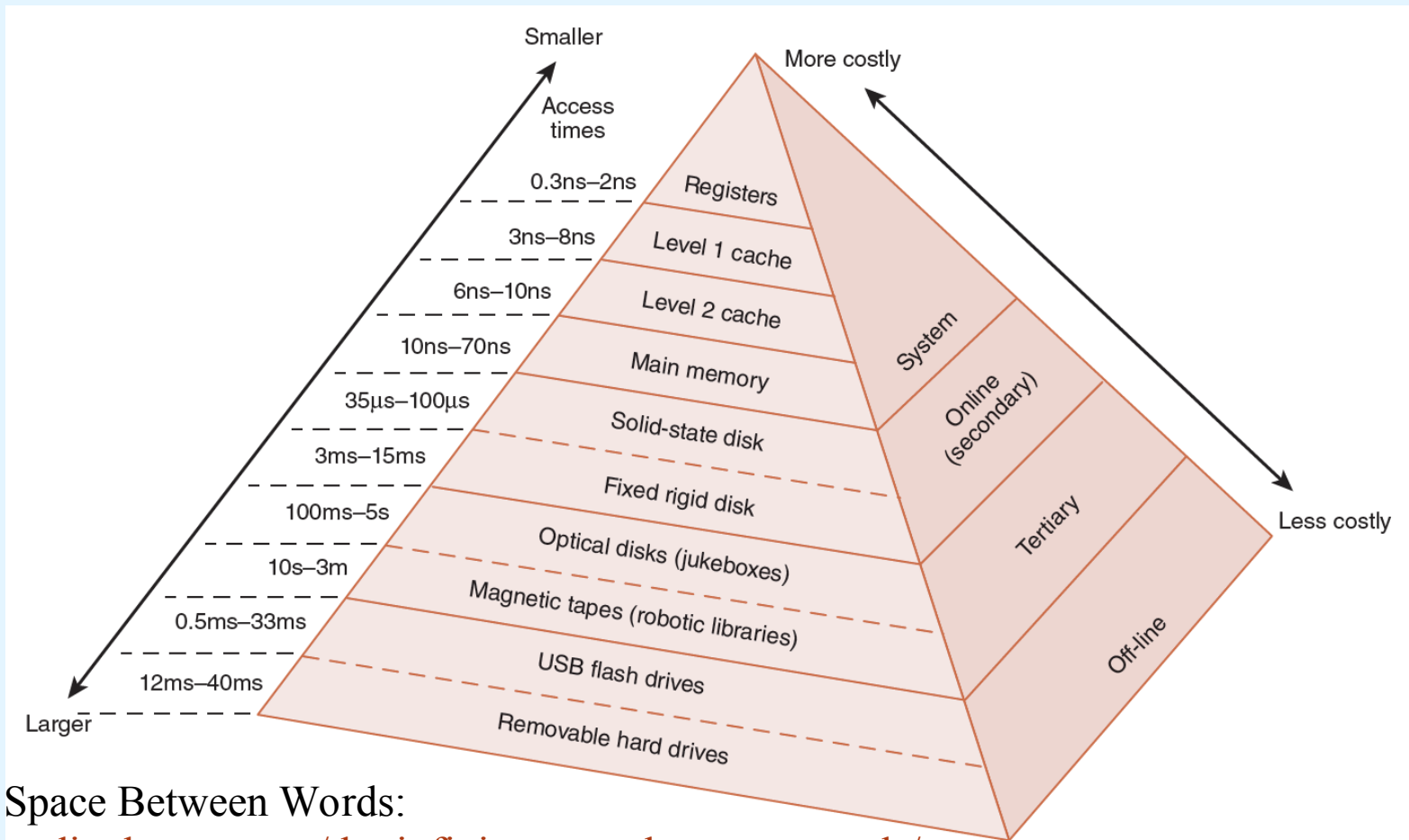


- conhecer a *hierarquia de memória* de um computador e compreender o impacto de cada *nível* no desempenho
- conhecer os tipos de memórias, componentes usados na sua construção, e a sua organização física e lógica
- compreender os conceitos subjacentes às *caches*
- saber medir o impacto das *caches* no desempenho

5.1 Hierarquia de Armazenamento

- Níveis de Armazenamento

- organização: **velocidade, capacidade, custo, persistência**
 - memória + (-) rápida é + (-) pequena, + (-) cara e + (-) volátil



The Infinite Space Between Words:

<https://blog.codinghorror.com/the-infinite-space-between-words/>

5.1 Hierarquia de Armazenamento

- Níveis de Armazenamento



- num sistema de computação, os vários meios de armazenamento são organizados hierarquicamente
- os meios de armazenamento + rápidos e de < capacidade são mantidos na / próximo da CPU
- meios mais lentos e de maiores dimensões são acedidos através de barramento externos à CPU
- suportes de armazenamento persistente e de capacidades elevadas, como discos e bandas magnéticas, ficam ainda mais “distantes” da CPU
- abaixo dos registos, aceder a um **item** (instrução ou dado) implica aceder a um **bloco** que o contém

5.1 Hierarquia de Armazenamento

- Acesso Hierárquico



- para a CPU utilizar um item, esse item tem que estar num **registo**; se não estiver, tem que ser copiado para lá, pelo que a CPU pede-o às memórias mais próximas, que são, tipicamente, as **memórias cache da CPU**
- se o item não for encontrado numa linha (bloco) das caches da CPU, é solicitado à **memória principal (MP)**
- se o item não for encontrado num bloco da MP, é pedido ao nível de armazenamento **secundário (SSD/Disco)**
- e assim sucessivamente, se necessário descendo aos níveis de armazenamento **terciário** ou até **quaternário**
- uma vez localizado, o item (e seus vizinhos no mesmo bloco) são copiados pela hierarquia acima (cada nível irá comportar-se como uma *cache* do nível acima dele)

5.1 Hierarquia de Armazenamento

- Acerto, Taxa de Acerto, Tempo de Acerto

- **hit** (acerto/sucesso) num nível **n**:
 - o item pretendido encontra-se no nível **n**
 - se o item foi pedido pelo nível acima **n-1**, é-lhe entregue
 - a entrega pode ser apenas do item (e.g., quando a cache da CPU entrega à CPU um item (dado/instrução) de uma linha da cache)
 - ou pode incluir um bloco ao qual o item pertence (e.g., quando a memória principal entrega um bloco de memória à cache da CPU)
- **hit rate** (taxa de acerto/sucesso) num nível:
 - percent. de vezes que um item é encontrado nesse nível
- **hit time** (tempo de acerto/sucesso) num nível:
 - tempo gasto para aceder a um item guardado nesse nível

5.1 Hierarquia de Armazenamento

- Insucesso, Tx. de Insuc., Penalidade por Insuc.

- **miss** (insucesso) num nível **n**: o item pretend. não está no nível **n**
 - o item tem de ser pedido ao nível abaixo **n+1**
 - pode-se pedir apenas o item (e.g., quando a CPU pede um item (dado / instrução) à sua cache)
 - ou pode-se pedir o bloco ao qual o item pertence (e.g., quando a cache da CPU pede à memória principal o bloco com o item)
- **miss rate** (tx. insucesso) num nível: **miss rate** = $1 - \text{hit rate}$
 - percentagem de vezes que um item NÃO está nesse nível
- **miss penalty** num nível **n**: tempo gasto para resolver um **miss**
 - inclui o tempo i) de pesquisa do item no nível abaixo **n+1** e ii) de cópia do item (ou do bloco envolvente) para o nível **n**
 - após o item/bloco omissso ter sido copiado para o nível **n**, i) pode ser imediatamente consumido pelo solicitante original, ii) ou este pode repetir o acesso ao nível **n** (que agora resultará em **hit**)

5.1 Hierarquia de Armazenamento

- (Princípio da) Localidade de Referências

- *localidade **temporal*** :
 - itens acedidos **recentemente** tendem a ser novamente acedidos num futuro próximo (pelo menos durante um período limitado, após o que a localidade pode mudar)
- *localidade **espacial*** :
 - os acessos a um meio de armazenamento tendem a **concentrar-se em certas zonas** desse meio (e.g., linha de cache, bloco de memória principal, bloco de disco)
- *localidade **sequencial*** :
 - quando se acede **sequencialmente** a itens armazenados **contiguamente** (e.g., acesso a células seguidas de um array, execução sequencial de instruções sem saltos)

5.1 Hierarquia de Armazenamento

- Tempo de Acesso Efetivo

- o desempenho do armazenamento hierárquico é medido pelo **Tempo de Acesso Efetivo** (EAT)
- o EAT corresponde a uma **média** pesada que tem em conta a taxa de sucesso (**hit rate**) e os tempos de acesso relativos de níveis de armazen. sucessivos
- uma expressão para o EAT deve levar em conta se há ou não acessos simultâneos a vários níveis
 - acesso **sobreposto**: um acesso a um nível **n** ocorre em simultâneo com um acesso ao nível inferior (**n+1**); havendo **hit** no nível **n**, aborta-se o acesso ao nível **n+1**; se houver **miss**, então poupou-se tempo no acesso ao nível **n+1**
 - acesso **sequencial**: um nível **n+1** só é acedido se após se consultar o nível **n** essa consulta resultar em **miss**

5.1 Hierarquia de Armazenamento

- Tempo de Acesso Efetivo



- **EAT de uma hierarquia com 2 níveis**

- acesso **sobreposto**:

$$EAT_2 = H1 \times T1 + (1-H1) \times T2$$

- acesso **sequencial**:

$$EAT_2 = H1 \times T1 + (1-H1) \times (T1 + T2)$$

- ***H1*** é a taxa de sucesso (*hit rate*) do 1º nível da hierarquia
 - a taxa do 2º nível (***H2***), não mostrada, assume-se 100%
- ***T1*** e ***T2*** são os tempos de acesso do 1º e 2º níveis

5.1 Hierarquia de Armazenamento

- Tempo de Acesso Efetivo



- **EAT de uma hierarquia com 2 níveis (exemplo):**
 - com um tempo de acesso $T_1=10\text{ns}$ à *cache* da CPU, uma taxa de sucesso $H_1=99\%$ no acesso a essa *cache*, e um tempo de acesso $T_2=200\text{ns}$ à memória principal, o EAT é:

acesso sobreposto



$$\text{EAT} = \underbrace{0.99(10\text{ns})}_{\text{cache hit}} + \underbrace{0.01(200\text{ns})}_{\text{cache miss}} = 9.9\text{ns} + 2\text{ns} = 11.9\text{ ns}$$

$$\text{EAT} = \underbrace{0.99(10\text{ns})}_{\text{cache hit}} + \underbrace{0.01 \left(\overbrace{10\text{ns}}^{\text{check cache}} + \overbrace{200\text{ns}}^{\text{go to main memory}} \right)}_{\text{cache miss}} = 9.9\text{ns} + 2.1\text{ns} = 12\text{ns}$$

acesso
sequencial



- as expressões do EAT com dois níveis podem generalizar-se para um qualquer número de níveis de memória ... (assumindo que a tx. de acerto do último nível é 100%)

5.1 Hierarquia de Armazenamento

- Tempo de Acesso Efetivo



- **EAT de uma hierarquia com 2 vs 3 vs 4 níveis**

acesso sobreposto

$$EAT_2 = H1 \times T1 + (1-H1) \times T2$$

$$EAT_3 = H1 \times T1 + (1-H1) \times [H2 \times T2 + (1-H2) \times T3]$$

$$EAT_4 = H1 \times T1 + (1-H1) \times [H2 \times T2 + (1-H2) \times [H3 \times T3 + (1-H3) \times T4]]$$

acesso sequencial

$$EAT_2 = H1 \times T1 + (1-H1) \times (T1 + T2)$$

$$EAT_3 = H1 \times T1 + (1-H1) \times [H2 \times (T1 + T2) + (1-H2) \times (T1 + T2 + T3)]$$

$$EAT_4 = H1 \times T1 + (1-H1) \times [H2 \times (T1 + T2) + (1-H2) \times [H3 \times (T1 + T2 + T3) + (1-H3) \times (T1 + T2 + T3 + T4)]]$$

checkpoint: exercícios EAT1, EAT2 e EAT3

5.2 Tipos de Memória



- Dois tipos principais de memória: **RAM e ROM**
- **RAM** - memória de acesso aleatório (leitura e escrita)
 - designa, geralmente, a *memória principal ou primária* (MP); as *caches* são tb baseadas em RAM, mas de tipo especial
 - pode ser modificada (suporta leitura e escrita), mas é volátil (perde o seu conteúdo na ausência de corrente)
 - **RAM dinâmica (DRAM): para memória principal**
 - **1 transístor + 1 condensador (*)** por bit; condensador perde a carga => necessidade de recarga periódica
 - muitas variantes: multibank (MDRAM), fast-page mode (FPM), extended data out (EDO), synchronous dynamic RAM (**SDRAM**), double data rate (**DDR**) SDRAM, rambus DRAM (RDRAM), direct rambus (DR) DRAM

(*)
DRAM
1-bit cell

5.2 Tipos de Memória



- **RAM (cont.)**
 - **RAM estática (SRAM):** para memórias de tipo *cache*
 - **>= 4 transístores (*)** por bit; mantêm o estado atual (valor 0/1), desde que alimentados eletricamente (acesso mais rápido por não ser necessário refresh)
 - variantes: [a]synchronous SRAM, pipeline-burst SRAM
 - **DRAM vs SRAM**
 - DRAM: - componentes => + barata, > densidade de bits, > capacidade, necessita de + corrente (refresh periódico)
 - SRAM: + rápida e + cara que DRAM

(*)
SRAM
1-bit cell

RAM Explained - Random Access Memory: <https://www.youtube.com/watch?v=PVad0c2cljo>
(*) How does Computer Memory Work?: <https://www.youtube.com/watch?v=7J7X7aZvMXQ>
Static RAM and Dynamic RAM Explained: <https://www.youtube.com/watch?v=0rNEtAz3wJQ>
SRAM vs DRAM: The Speed Difference between Cache and RAM:
<https://www.youtube.com/watch?v=ammQYLcyebA>

5.2 Tipos de Memória



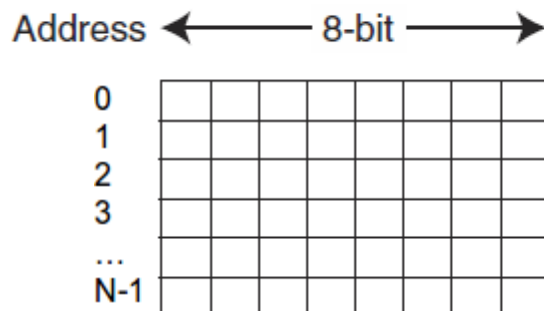
- **ROM** – memória apenas de leitura / escrita limitada
 - usada para armazenar dados (semi)permanentes (e.g., na BIOS), que se mantêm quando o sistema é desligado
 - variantes:
 - **ROM** (Read-Only Memory) – apenas de leitura
 - **PROM** (*programmable* ROM) – com fusíveis que podem ser “queimados” para uma só reprogramação
 - **EPROM** (*erasable* PROM) – PROM reprogramável (com luz ultra-violeta) várias vezes (mas só totalidade do chip)
 - **EEPROM** (*electrically erasable* PROM) – reprogramável byte-a-byte e de forma mais fácil (só c/ campo elétrico)
 - **Flash** – EEPROM reprogramável por blocos (+ rápido)

5.3 Memória Principal

- Organização e Endereçamento

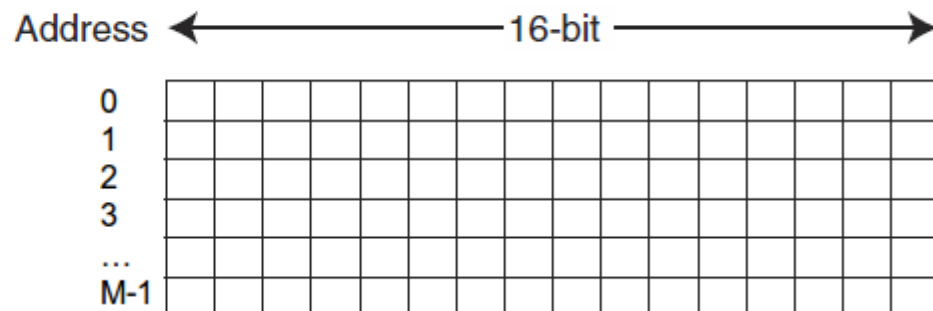
- **visão abstrata: Memória Principal = vetor** (*array*) de **células de memória** (ou simplesmente **células**)
- uma **célula** = a 8 (ou múltiplo de 8) **1-bit cell**
- cada célula tem um **endereço único** (inteiro ≥ 0)
- visão abstrata refinada: a MP é uma **matriz de bits**, de ordem **LxB** (**L** linhas de altura e **B** bits de largura)

a) **N 8-Bit Memory Locations**



memória N x 8 bits

b) **M 16-Bit Memory Locations**



memória M x 16 bits

5.3 Memória Principal

- Organização e Endereçamento



- um **byte** = 8 bits; uma **palavra** = 1 ou mais bytes
- **memória endereçável ao byte** (célula = byte)
 - cada byte possui um endereço único
 - ainda assim, as **instruções** podem ocupar ou manipular uma palavra, e os **registos** podem alojar uma palavra
 - ou seja: a palavra de memória e a palavra da arquitetura do conjunto de instruções podem ter dimensões diferentes
 - isto é o mais comum na maioria dos computadores atuais
 - o byte com < endereço determina o endereço da palavra
- **memória endereçável à palavra** (célula = palavra)
 - cada palavra possui um endereço único
- endereços e instruções costumam caber numa palavra

5.3 Memória Principal

- Organização e Endereçamento

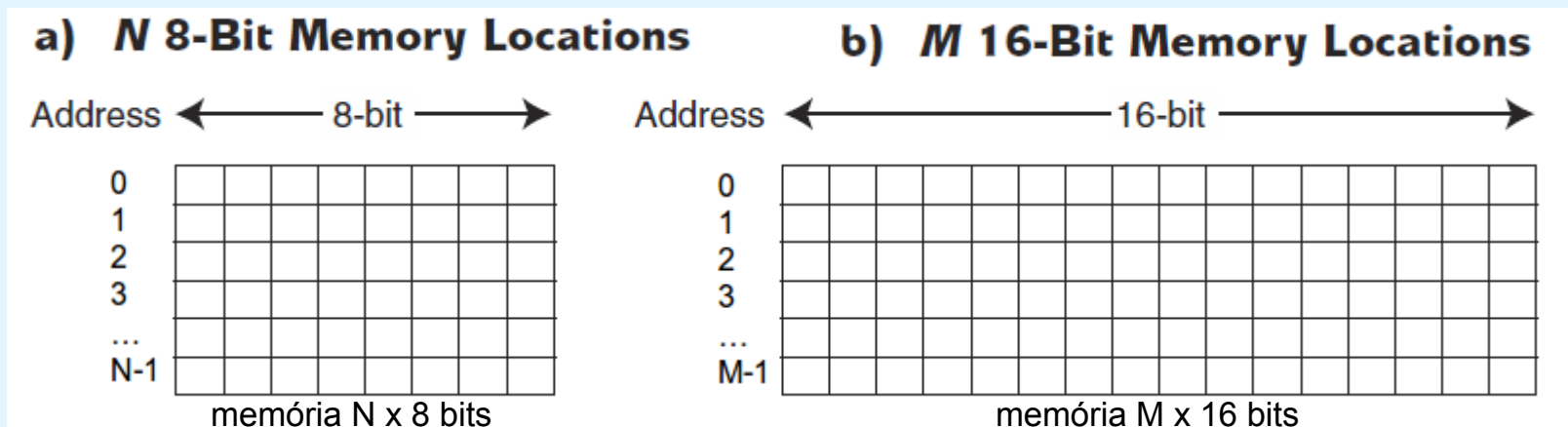


- quando a memória é endereçável ao byte, e a ISA é endereçável à palavra (de ≥ 2 bytes), torna-se necessário lidar com a questão do **alinhamento**
- exemplo: ler uma palavra de 32 bits numa memória endereçável ao byte $\Rightarrow$ endereço seja múltiplo de 4
- exceção: algumas arquiteturas permitem que certas instruções façam acessos à memória não-alinhados
 - ou seja, os endereços aos quais se pretende aceder não são necessariamente múltiplos do tamanho da palavra
 - consequência: pode implicar ler duas palavras para aceder ao equivalente a uma (metade do que se pretende está numa palavra, e o resto na seguinte)

5.3 Memória Principal

- Organização e Endereçamento

- numa memória com **C** células
 - as células são numeradas (endereços) de **0** a **C-1**
 - o nº de bits dos endereços das células é **$\log_2 C$**
 - **exemplo:**
 - a) célula=1 byte (endereço/ ao byte); **N** células; endereços **0** a **N-1** de **$\log_2 N$** bits
 - b1) célula=1 byte (endereço/ ao byte); **2M** células; endereços **0** a **2M-1** de **$\log_2 2M$** bits
 - b2) célula=2 bytes (endereço/ à palavra); **M** células; endereços **0** a **M-1** de **$\log_2 M$** bits

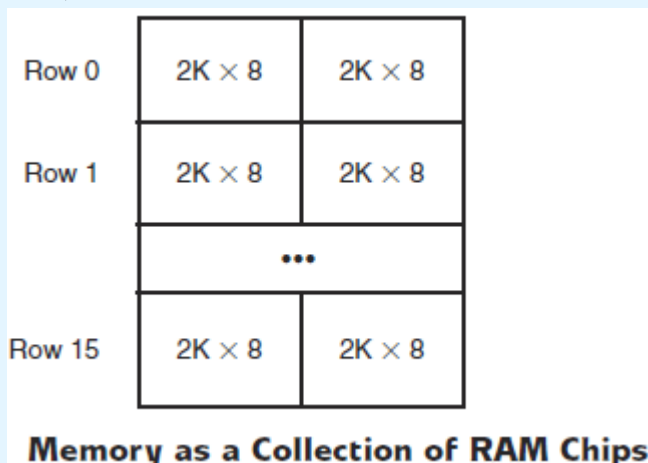


checkpoint: exercícios MP1 a MP4

5.3 Memória Principal

- Organização e Endereçamento

- **visão física:** uma **memória** baseia-se em um ou mais **módulos**, constituídos por um ou mais **chips**
- módulos e chips são também memórias (+ pequenas), logo descritos pela notação **altura x largura (LxB)**
- **exemplo 2:** memória **32K x 16**, à custa de 16 módulos de **2K x 16**, realizados com pares de chips **2K x 8**



módulo (ram stick) com 8 chips
[<https://en.wikipedia.org/wiki/DIMM>]

5.3 Memória Principal

- Organização e Endereçamento



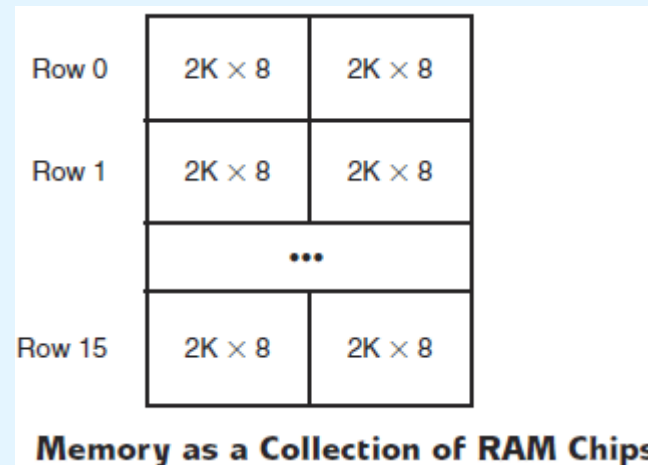
- **mapeamento visão abstrata (lógica) → visão física:**
 - associa cada **endereço da memória** a um **módulo** e a um **deslocamento** ou **linha** dentro desse módulo
 - a sequência de bits de um **endereço** é dividida em duas
 - uma, que indica o **módulo** onde o endereço foi mapeado
 - outra, que indica o **deslocamento** dentro do módulo
 - desta subdivisão decorre que:
$$\#BITS(\text{endereço}) = \#BITS(\text{módulo}) + \#BITS(\text{deslocamento})$$
 - adicionalmente:
$$\#BITS(\text{endereço}) = \log_2(\#LINHAS(\text{memória}))$$
$$\#BITS(\text{módulo}) = \log_2(\#MÓDULOS(\text{memória}))$$
$$\#BITS(\text{deslocamento}) = \log_2(\#LINHAS(\text{módulo}))$$

5.3 Memória Principal

- Organização e Endereçamento



- mapeamento visão abstrata (lógica) → visão física:
 - exemplo 2 (cont.):



- com endereçamento à palavra de 16 bits, há 32K endereços
- cada um desses endereços escreve-se com $\log_2(32K) = 15$ bits
- destes 15 bits, 4 determinam o módulo e 11 o deslocamento
 - 16 módulos identificam-se com $\log_2(16) = 4$ bits
 - 2K linhas de módulo identificam-se com $\log_2(2K) = 11$ bits
(nota: linha de módulo = deslocamento)

5.3 Memória Principal

- Organização e Endereçamento



- **mapeamento visão abstrata (lógica) → visão física:**
 - **High-Order Interleaving (HOI):** os bits mais significativos do endereço especificam o módulo, e os bits menos significativos indicam o deslocamento dentro do módulo
 - como os endereços de memória são mapeados por ordem (sequencialmente) pelos vários módulos, o acesso a uma sequência de palavras (que cabe num módulo) é *serializado*
 - **Low-Order Interleaving (LOI):** os bits menos significativos do endereço especificam o módulo, e os bits mais significativos indicam o deslocamento dentro do módulo
 - como os endereços de memória são mapeados rotativamente (de forma circular) pelos vários módulos, o acesso a palavras de uma sequência pode ser feito em paralelo (desde que as palavras estejam em diferentes módulos)

5.3 Memória Principal

- Organização e Endereçamento

- mapeamento visão abstrata (lógica) → visão física:
 - exemplo 2 (cont.): **checkpoint: exercícios MP5 a MP9**

#BITS(address) = 15; #BITS(module) = 4; #BITS(offset) = 11; 30722 (10) = 111 1000 0000 0010 (2)
 HOI: 111 1000 0000 0010 => Module = 1111 (2) = 15 (10), Offset = 000 0000 0010 (2) = 2 (10)
 LOI: 111 1000 0000 0010 => Module = 0010 (2) = 2 (10), Offset = 111 1000 0000 (2) = 1920 (10)

	Logical Address	Physical Address				
		HOI mapping	LOI mapping	Offset	Module	
32K x 16	0	0	0	0	0	2K x 16
	1	1	16	1		
	2	2	32	2		
	...	...	...	...		
	2047	2047	32752	2047		
	2048	2048	1	0	1	2K x 16
	2049	2049	17	1		
	2050	2050	33	2		
	...	...	...	...		
	4095	4095	32753	2047		
	...					...
	30720	30720	15	0	15	2K x 16
	30721	30721	31	1		
	30722	30722	47	2		
	...	...	...	...		
	32767	32767	32767	2047		

5.4 Cache da CPU

CPU Cache Explained - What is Cache Memory?
<https://www.youtube.com/watch?v=yi0FhRqDJfo>

- acelera o acesso a instruções/dados, mantendo perto da CPU os blocos da MP usados mais recentemente
- a *cache* é muito mais pequena que a MP, mas o seu tempo de acesso é também muito mais reduzido
- ao contrário da **MP**, acedida com base em **endereços**, uma **cache** é acedida pelo **conteúdo** (é uma CAM)
 - *CAM = content adressable memory*
(memória endereçável por conteúdo)
- o conteúdo em causa corresponde a **campos**
 - “subconjunto(s) dos bits de um endereço da MP”
 - permitem identificar linhas/blocos precisos da *cache*
 - acessos à MP => acessos a linhas/blocos completos

5.4 Cache da CPU

- Utilização Típica



- **campos de um endereço da MP visto pela cache:**
 - **tag** [, **block/set**], **word/offset** [...] significa opcional)
- **utilização dos campos de um endereço da MP:**
 - o **block** (**set**) indica o bloco da *cache* (ou *conjunto* de blocos), onde o item pretendido está (**hit**) ou onde deveria estar (**miss**)
 - o bloco da *cache* pode ser válido ou não (**miss**), de acordo com o valor de um **bit de validade** guardado no próprio bloco
 - se o bloco for válido, uma **tag** extraída do endereço é comparada com uma **tag** guardada no próprio bloco
 - se as tags forem iguais (**hit**), o conteúdo da palavra referenciada pelo campo **offset** é extraído do bloco da cache e dado à CPU
 - se as tags forem diferentes (**miss**), o bloco em falta é pedido à memória principal (onde é identificado por **tag** [+ **block/set**])

5.4 Cache da CPU

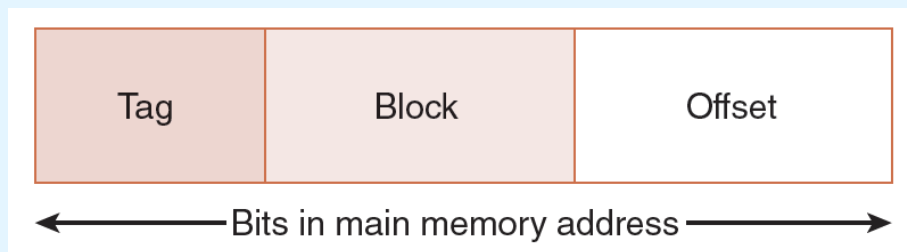
- Esquemas de Correspondência MP → Cache

- a divisão de um endereço da MP em **campos** dá origem a uma correspondência de “**muitos-para-um**”
 - diferentes blocos da MP são associados ao mesmo bloco da *cache*; será o valor da etiqueta (*tag*) no bloco da *cache* que permite saber que bloco da MP foi para lá copiado
- **esquemas de correspondência:**
 - diferentes formas de dividir um endereço da MP em **campos** + algoritmo específico de gestão da cache
 - principais esquemas de correspondência/mapeamento:
 - mapeamento direto
 - mapeamento associativo
 - mapeamento associativo por conjuntos

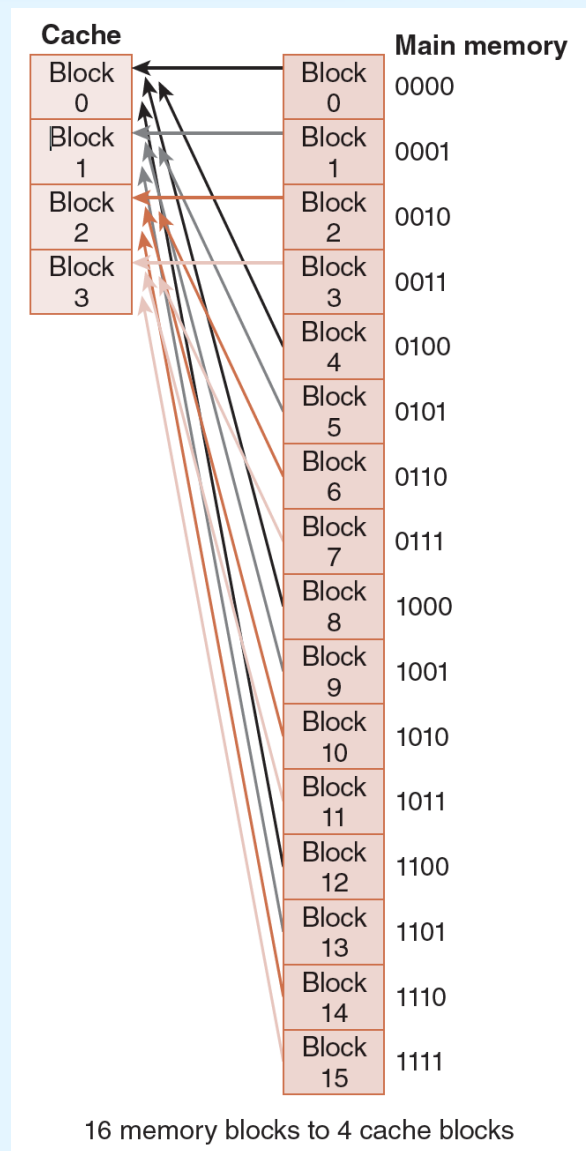
5.4 Cache da CPU

- Correspondência Direta (*direct mapped cache*)

- é o esquema mais simples



- numa *cache* com N blocos, o “bloco X da memória principal” corresponde ao “bloco da *cache* $Y = X \bmod N$ ” (i.e., $Y = X \% N$)
- exemplo: se $N=4$ blocos, no bloco 0 da *cache* poderão ser guardadas cópias dos blocos 0, 4, 8, 12, ... da memória principal



5.4 Cache da CPU

- Correspondência Direta (*direct mapped cache*)

- **anatomia da *cache*** (exemplo):

Block	Tag	Data	Valid
0	00000000	words A, B, C,...	1
1	11110101	words L, M, N,...	1
2	-----		0
3	-----		0

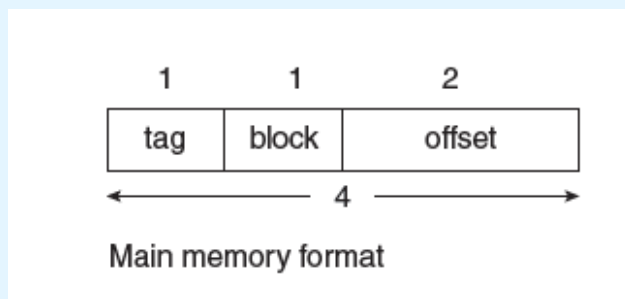
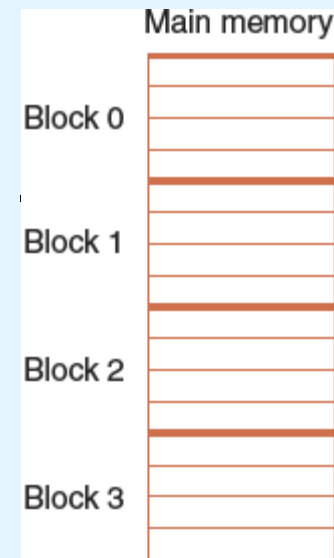
- o bloco 0 da *cache* contém várias palavras da memória principal (A,B,C,...), identificadas pela tag 00000000
- o bloco 1 da *cache* contém várias palavras da memória principal (L,M,N,...) identificadas pela tag 11110101
- os outros blocos não são válidos ...

5.4 Cache da CPU

- Correspondência Direta (*direct mapped cache*)

- **dimensionamento dos campos:**

- o tamanho de cada campo de um endereço da memória principal depende do tamanho da *cache*
- **exemplo 1:** i) MP com 16 palavras, ii) *cache* com 2 blocos, iii) cada bloco armazena 4 palavras
 - iii) $\Rightarrow \log_2 4 = 2$ bits para endereçar as palavras
 - ii) $\Rightarrow \log_2 2 = 1$ bits para endereçar os blocos
 - i) $\Rightarrow$ como $\log_2 16 = 4$, sobram $4 - (2 + 1) = 1$ bits para a *tag*
 - número de blocos da MP = $2^{(\#BITS(tag) + \#BITS(block))} = 4$
= num. de palavras da MP / num. de palavras do bloco = $16/4$



5.4 Cache da CPU

- Correspondência Direta (*direct mapped cache*)

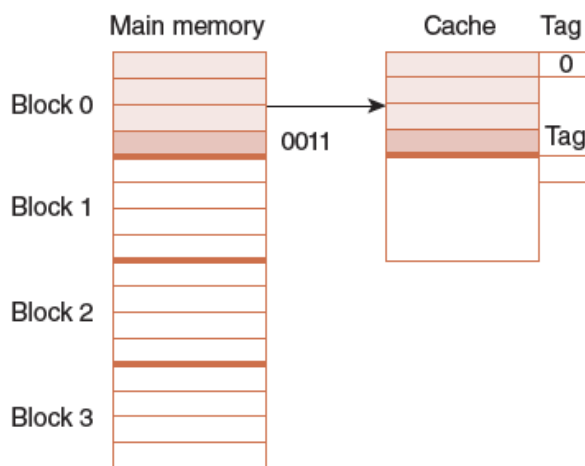
- dimensionamento dos campos (cont.):
 - exemplo 1 (cont.):

0	0	1 1
tag	block	offset

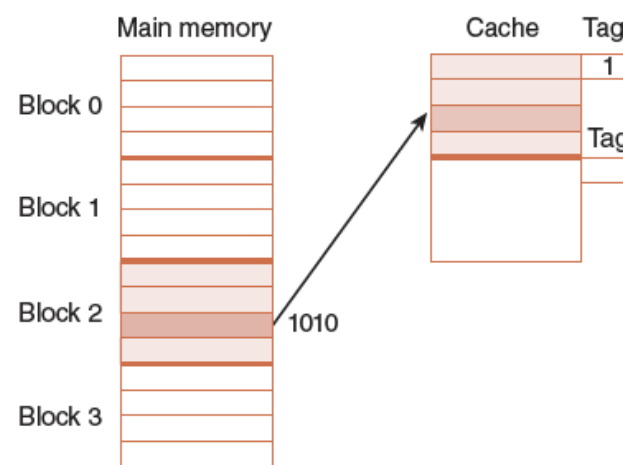
The address 0011 partitioned into fields

1	0	1 0
tag	block	offset

The address 1010 partitioned into fields



Mapping of block containing
Address 0011 = 0x3



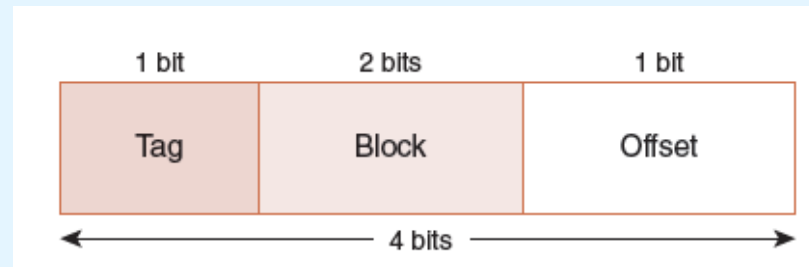
Mapping of Block Containing
Address 1010 = 0xA

5.4 Cache da CPU

- Correspondência Direta (*direct mapped cache*)

- **dimensionamento dos campos (cont.):**
 - **exemplo 2:** i) MP formada por 16 palavras, ii) cache com 4 blocos, iii) cada bloco armazena 2 palavras
 - iii) => necessários $\log_2 2 = 1$ bits para endereçar as palavras
 - ii) => necessários $\log_2 4 = 2$ bits para endereçar os blocos
 - i) => como $\log_2 16 = 4$, sobram $4 - (1+2) = 1$ bits para a tag
 - número de blocos da MP = $2^{(\#BITS(tag)+\#BITS(block))} = 8$
= num. de palavras da MP / num. de palavras do bloco = $16/2$

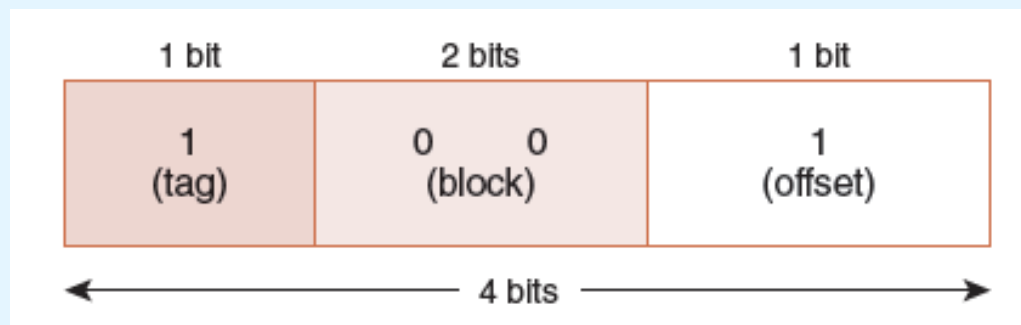
Main Memory	Maps To	Cache
(000) Block 0 (addresses 0x0, 0x1)	→	Block 0 (00)
(001) Block 1 (addresses 0x2, 0x3)	→	Block 1 (01)
(010) Block 2 (addresses 0x4, 0x5)	→	Block 2 (10)
(011) Block 3 (addresses 0x6, 0x7)	→	Block 3 (11)
(100) Block 4 (addresses 0x8, 0x9)	→	Block 0 (00)
(101) Block 5 (addresses 0xA, 0xB)	→	Block 1 (01)
(110) Block 6 (addresses 0xC, 0xD)	→	Block 2 (10)
(111) Block 7 (addresses 0xE, 0xF)	→	Block 3 (11)



5.4 Cache da CPU

- Correspondência Direta (*direct mapped cache*)

- **dimensionamento dos campos (cont.):**
 - **exemplo 2 (cont):** endereço 9(10) = 1**00**1(2)

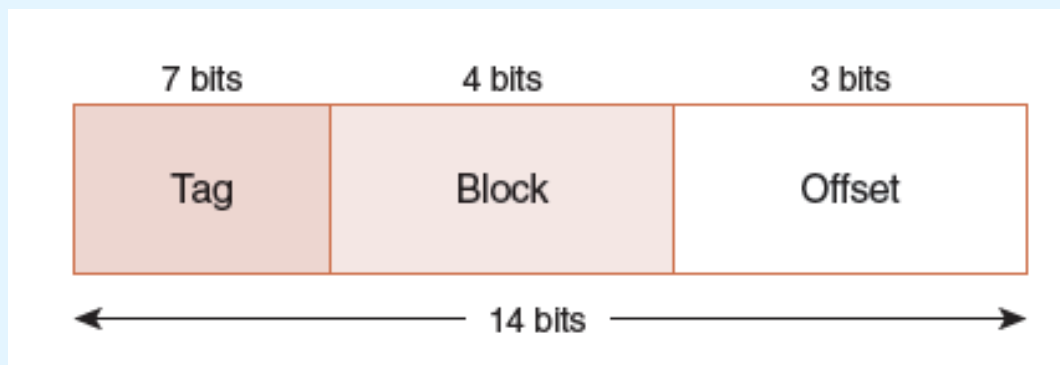


- os endereços 0**00**0 (2), 0**00**1 (2), 1**00**0 (2) e 1**00**1 (2) (ou seja 0,1,8 e 9) serão mapeados sempre no bloco **00** da *cache*
- um primeiro acesso a 0(10) viabiliza acesso imediato a 1(10)
- um primeiro acesso a 8(10) viabiliza acesso imediato a 9(10)
- qual o efeito de uma sequência de acessos 0/1, 8/9, 0/1, 8/9, ...?

5.4 Cache da CPU

- Correspondência Direta (*direct mapped cache*)

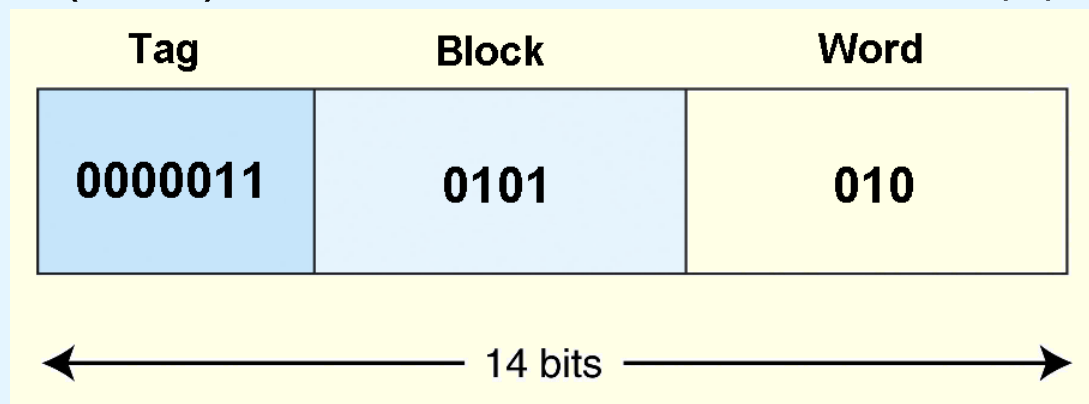
- **dimensionamento dos campos (cont.):**
 - **exemplo 3:** i) MP com $16K=2^{14}$ palavras, ii) cache com 16 blocos, iii) cada bloco armazena 8 palavras
 - iii) => necessários 3 bits para endereçar as palavras
 - ii) => necessários 4 bits para endereçar os blocos
 - i) => sobram $14 - (3+4) = 7$ bits para o campo *tag*
 - número de blocos da MP = $2^{(\#BITS(tag)+\#BITS(block))} = 2^{11} = \text{num. palav. MP} / \text{num. palav. bloco} = 2^{14}/2^3$



5.4 Cache da CPU

- Correspondência Direta (*direct mapped cache*)

- **dimensionamento dos campos (cont.):**
 - **exemplo 3 (cont):** endereço 00000110101010 (2) = 1AA (16)



- o endereço inicial de um bloco é sempre um **múltiplo de 8**
- a palavra 1AA pertence ao bloco com as palavras **1A8** a 1AF; o acesso à palavra 1AA copia para a *cache* essas 8 palavras
- acesso imediato a qq das 8 palavras é satisfeito pela *cache*
- qual o efeito de uma sequência de referências 1AA,3AA,... ?

5.4 Cache da CPU

- Correspondência Direta (*direct mapped cache*)

- **vantagens:**
 - esquema simples, de fácil implementação ...
 - ... o que o torna economicamente atrativo
- **desvantagens:**
 - a correspondência fixa entre “blocos da MP” e “blocos da *cache*” faz com que, para certas sequências de acesso à MP, a *cache* não seja devidamente rentabilizada
 - ou seja, existe um desfasamento entre a localidade de referências e a política de substituição dos blocos da *cache*
 - necessidade de esquemas que resolvam este problema

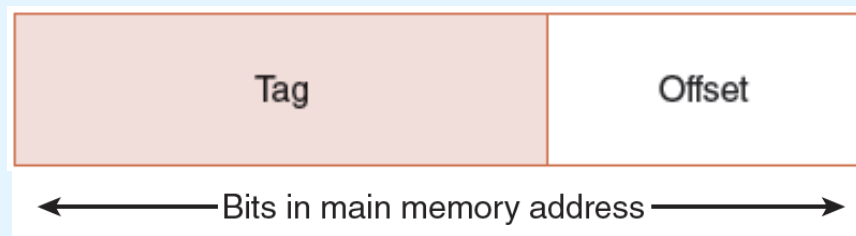
checkpoint: simulador Camera; exercícios
CACHE1, CACHE2, CACHE3/EAT4

5.4 Cache da CPU

- Cache Totalmente Associativa (*fully associative cache*)

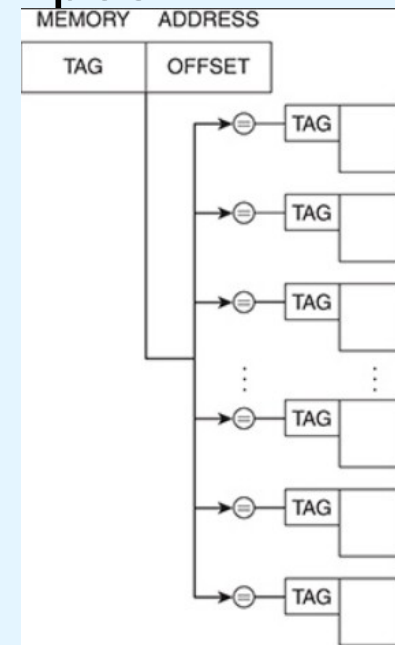
- **princípios:**

- um bloco da MP pode-se associar a qq bloco da *cache*
- um endereço da MP divide-se apenas em 2 campos



- **pesquisa na *cache* por um bloco da MP:**

- implica pesquisar todos os blocos da *cache*
- *cache* construída com *memória associativa*
 - exige componentes especiais (mais caros)
 - *tags da cache* comparadas em paralelo c/ a tag do end.



5.4 Cache da CPU

- Cache Totalmente Associativa (*fully associative cache*)

- necessidade de um **algoritmo/política de substituição**:
 - quando a *cache* está totalmente preenchida, inserir um bloco novo na *cache* implica substituir um bloco ocupado
 - o bloco ocupado que será substituído chama-se *vítima*
 - existem várias formas de escolher o bloco *vítima*
 - *First-in-First-Out (FIFO)*
 - *Least Recently Used (LRU)*
 - *Least Frequently Used (LFU), etc.*
 - é também preciso considerar a necessidade de copiar o bloco *vítima* para a memória se tiver sido modificado

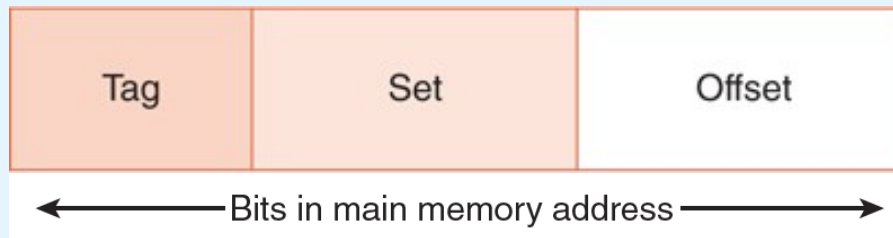
checkpoint:
simulador Camera;
exercícios CACHE4,
CACHE5

5.4 Cache da CPU

- Cache Associativa por Conjuntos (*set associative cache*)

- **princípios:**

- combinar correspondência *direta* com *associativa*
- um endereço da MP divide-se de novo em 3 campos



- o campo **set** identifica um conjunto de blocos na cache
 - os conjuntos têm um número de blocos comum
- um endereço será mapeado sempre no mesmo *set* ...
 - conceito importado do *mapeamento direto*
- ... mas poderá ser mapeado em qualquer bloco do *set*
 - conceito importado do *mapeamento associativo*

5.4 Cache da CPU

- Cache Associativa por Conjuntos (*set associative cache*)

- **princípios** (cont):

- “número de vias, N ” = “número de blocos por conjunto”
- exemplo de *cache* associativa por conjuntos com $N=2$ vias

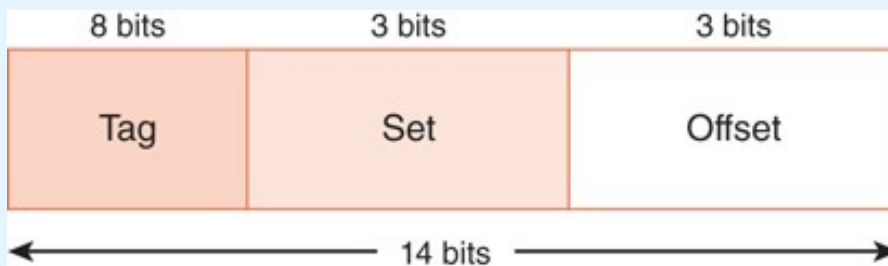
Set	Tag	Block 0 of set	Valid	Tag	Block 1 of set	Valid
0	00000000	Words A, B, C, ...	1	-----		0
1	11110101	Words L, M, N, ...	1	-----		0
2	-----		0	10111011	P, Q, R, ...	1
3	-----		0	11111100	T, U, V, ...	1

- N é um parâmetro importante no desenho de uma *cache*
 - uma *cache* de *mapeamento direto* representa um caso especial de *cache associativa por conjuntos*, em que $N=1$
- os campos **tag** e **offset** com função igual ao map. direto

5.4 Cache da CPU

- Cache Associativa por Conjuntos (set associative cache)

- **exemplo de dimensionamento** para $N=2$: i) MP com 2^{14} palavras, ii) cache com 16 blocos e iii) blocos de 8 palavras
 - iii) $\Rightarrow \log_2 8 = 3$ bits para endereçar as palavras
 - [ii) + $N=2$] $\Rightarrow$ há $16/2=8$ sets na cache
 - $\Rightarrow$ necessários $\log_2 8 = 3$ bits para endereçar os sets
 - i) $\Rightarrow$ sobram $14 - (3+3) = 8$ bits para o campo tag
 - número de blocos da MP = $2^{(\#BITS(tag)+\#BITS(set))} = 2^{11}$
= número palavras MP / número palavras bloco = $2^{14}/2^3$



checkpoint:
simulador Camera; exercícios
CACHE6 a CACHE8/EAT5

How Cache Works Inside a CPU:

<https://www.youtube.com/watch?v=zF4VMombo7U>

5.4 Cache da CPU

- Políticas de Substituição

- nas *caches associativa e associativa por conjuntos* são necessárias **políticas de substituição de blocos**
 - *cache totalmente associativa*, com K blocos:
 - qualquer dos K blocos é, *a priori*, candidato a vítima
 - *cache associativa por conjuntos*, de N vias:
 - qualquer dos N blocos de um conjunto é vítima potencial
- uma política **ótima** seria capaz de “prever o futuro”, determinando que blocos não serão mais necessários (ou não serão pelo maior período de tempo possível)
- apesar de ser impossível implementar uma política ótima, é interessante utilizá-la para analisar o desempenho de outros algoritmos de substituição

5.4 *Cache* da CPU

- Políticas de Substituição

- a escolha de uma política de substituição depende do tipo de localidade que se pretende otimizar (normalmente interessa a localidade temporal)
- abordagem **LRU** ($\equiv$ **Least Recently Used**)
 - “toma nota” do momento em que cada bloco foi acedido pela última vez, de forma a se substituir aquele que não tiver sido usado há mais tempo
 - desvantagem: > complexidade temporal e espacial
 - manter o historial de cada bloco exige bits adicionais e tempo para os atualizar (tornando a *cache* + lenta)

5.4 Cache da CPU

- Políticas de Substituição

- abordagem **FIFO** (*first-in, first-out*)
 - o bloco que se encontra na *cache* há mais tempo, (tenha sido usado recentemente ou não), é a vítima
 - abordagem + simples que a LRU (a mesma sobrecarga espacial, mas menor sobrecarga temporal)
- **substituições aleatórias**
 - vítima escolhida “à sorte” (a abordagem + simples)
 - tende a evitar situações de *thrashing* (um bloco X é expulso e substituído por outro, mas logo de seguida é necessário voltar a trazer o bloco X para a *cache*)

Cache Replacement Policies: https://www.youtube.com/watch?v=P_UYI23DddU

5.4 Cache da CPU

- Ineficácia da Cache



- as *caches* só são efetivas se os programas exibem *localidade*
- **exemplo de localidade deficiente no acesso a arrays 2D:**
 - diferentes linguagens guardam arrays 2D na memória de forma diferente: C → por linha ; Fortran → por coluna
 - se um programa em C acede a um array 2D por linha, uma ou mais linhas podem caber na cache; após o miss do 1º acesso, os acessos restantes serão satisfeitos pela cache
 - se o mesmo programa aceder por coluna, saltará entre linhas na memória, podendo provocar misses sucessivos
 - exemplo: array 5x4; cache comporta uma só linha do array; acesso por linha = 5 misses; acesso por coluna = 20 misses

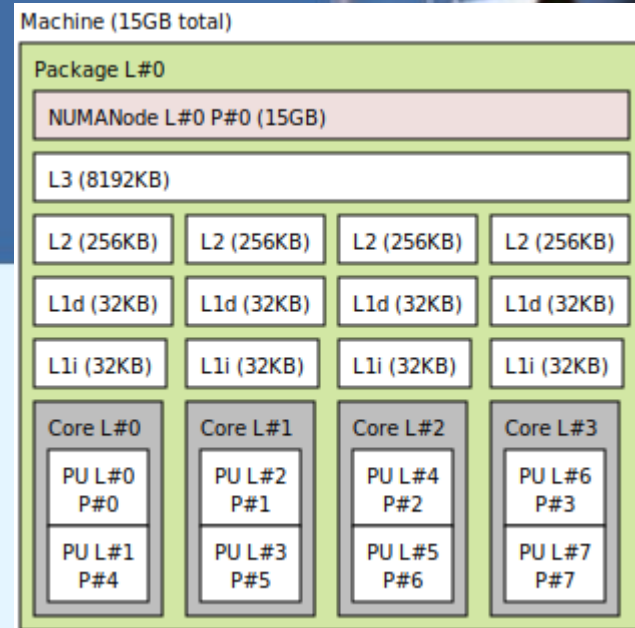
compiladores de C: row-major

compiladores de Fortran: column-major

5.4 Cache da CPU

- Ineficácia da Cache

- exemplo de localidade deficiente no acesso a arrays 2D - cenário real:
 - output de `lstopo` num sistema Linux: (em Windows, executar a app CPU-Z)
 - para garantir que um array 2D não cabe na cache maior (L3), o array terá de ocupar mais de 8192KB = 8MB; um array 2D de 4096x4096 células de 1 byte cada ocupará um total de 16 MB
 - o acesso ao array **por linha** é 4 vezes mais rápido que **por coluna**:



```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    char *a=(char*)malloc(4096*4096);
    for (int i=0; i<4096; i++)
        for (int j=0; j<4096; j++)
            a[i*4096+j]=0; // access by line: for each line, iterate all columns
            //a[j*4096+i]=0; // access by column: for each column, iterate all lines
}
```

```
$ time ./exemplo.exe
real 0m0,058s

$ time ./exemplo.exe
real 0m0,210s
```


5.4 Cache da CPU

- Políticas de Escrita



- as *políticas de substituição* devem também ter em conta os blocos “sujos” (***dirty blocks***) – blocos que foram modificados durante a sua estadia na cache
 - os *dirty blocks* têm de se copiar para a memória principal (seguindo uma ***política de escrita***), antes de substituídos
- política de **escrita imediata** (*write through*)
 - a *cache* e a memória principal são atualizadas em simultâneo sempre que há uma operação de escrita
- política de **escrita adiada** (*write back* ou *copy back*)
 - a MP só é atualizada quando o bloco é substituído

5.4 Cache da CPU

- Políticas de Escrita



- **(des)vantagens da escrita imediata**
 - (-) a memória principal tem de ser atualizada cada vez que a *cache* é alvo de uma escrita, o que penaliza o desempenho se as escritas forem muito frequentes (todavia, a maioria dos acessos tende a ser de leitura)
 - (+) a *cache* e a MP são mantidas num estado coerente
- **(des)vantagens da escrita adiada**
 - (-) a MP nem sempre “concorda” com a *cache*, podendo gerar-se incoerências em sistemas concorrentes
 - (+) minimiza o tráfego entre a *cache* e a MP

CPU Cache Write Policies: <https://www.youtube.com/watch?v=wfVy85Dqiyc>

5.4 Cache da CPU

- Caches Unificadas vs Separadas

- até agora, as *caches* assumiram-se **unificadas** ou **integradas**, contendo quer instruções, quer dados
- muitos sistemas modernos utilizam *caches* **separadas** para as instruções e para os dados (*Harvard cache*)
- a separação de instruções e dados por diferentes *caches* explora melhor a localidade de cada categoria, mas à custa de maior complexidade (na prática, uma *cache* unificada maior produz os mesmos benefícios)

5.4 *Cache* da CPU

- *Caches* Multinível



- a maioria dos sistemas modernos utilizam *caches* de CPU organizadas numa **hierarquia de vários níveis** (L1/2/3)
- cada CPU-core tem uma *cache* L1 (8KB a 64KB), tipicamente separada, com tempos de acesso à volta de 4ns
- cada CPU-core tem também uma *cache* L2 (64KB a 2MB), de tipo unificado, com tempos de acesso de 15ns a 20ns
- quando existem, as *caches* L3 (2MB a 256MB) ou de nível superior, são partilhadas pelos vários CPU-cores e podem situar-se “*off-die*” (fora do chip que aloja os CPU-cores)
 - nota: em tempos, as *caches* L2 eram também off-die

5.4 Cache da CPU

- Caches Inclusivas vs Exclusivas



- outro aspeto a considerar é se o mesmo item (dado ou instrução) pode existir em mais de um nível de *cache*
- se a hierarquia de *caching* for **inclusiva**, podem existir cópias dos mesmos itens em mais de um nível
 - numa hierarquia de caching **estritamente inclusiva**, garante-se que todo o conteúdo de um nível superior está replicado no nível imediatamente inferior (tacitamente assumido até agora)
- se a hierarquia de *caching* for **exclusiva**, **NÃO** podem existir cópias dos mesmos itens em mais de um nível
- cada opção implica compromissos diferentes em relação aos tempos de acesso, dimensão e complexidade da *cache*

Cache Hierarchy: How Modern CPU Caches Are Organized (L1, L2 and L3):

https://www.youtube.com/watch?v=7yrK_9PderQ

Conclusões

- num computador, o armazenamento organiza-se numa hierarquia, com os suportes de menor capacidade e tempo de acesso mais rápido no topo dessa hierarquia
- há memórias mais orientadas a leitura (ROM) e outras a leituras+escritas, assentes em tecnologias diferentes
- as memórias são construídas com módulos que agregam chips de igual capacidade; a capacidade total da memória resulta da capacidade agregada dos vários chips usados
- a distribuição dos endereços de uma memória pelos seus módulos pode ser feita de pelo menos duas formas (high-order interleaving e low-order interleaving)

Conclusões

- a memória principal organiza-se também em blocos de palavras, tendo todos os blocos a mesma capacidade
- uma *cache* faz corresponder blocos de memória principal a blocos da própria *cache*, com objetivo de reduzir o tempo de acesso a dados e instruções
- existem 3 tipos fundamentais de *cache*: de correspondência direta, associativa e associativa por conjuntos
- nas *caches* associativas e associativas por conjuntos, são necessárias políticas de substituição; nestas podem ser usados os algoritmos LRU, FIFO e LFU; é também necessário ter em conta a atualização de blocos alterados

Referências

- livro ECOA (6ª Edição) – Capítulo 4
(Organização da Memória e Endereçamento)
- livro ECOA (6ª Edição) – Capítulo 6
(Hierarquia da Memória e Memória Cache)
- <https://www.youtube.com/@BitLemonSoftware/videos>
(neste canal, ver também o vídeo **Cache Miss Types**
https://www.youtube.com/watch?v=YXkfmyb0_eA&t=1s)